

DESIGN

clickup_sync — Phase-0 Architecture & Design

Revision: 1 **Last modified:** 2026-06-09T00:00:00Z **Status:** DESIGN ONLY (Phase 0). NO implementation code, NO repo creation, NO ClickUp writes, NO main-tree source edits exist as a result of this document. Every "the engine will ..." / "MUST" statement below is a DESIGNED Phase-N outcome, not a current fact. **Authority:** Operator mandate (ClickUp bidirectional-sync initiative). Composes the HelixConstitution submodule §11.4 anti-bluff covenant family. **Design provenance:** this document + its sibling PLAN.md.

0. Honest scope & evidence boundary (§11.4.6 no-guessing)

Captured FACTS this design rests on (read-only, this worktree, 2026-06-09):

- **Workable-items model** — docs/Issues.md / docs/Fixed.md heading shape ## §X.Y [ATM-NNN] title with ****Status:**** + ****Type:**** lines; Issues_Summary.md / Fixed_Summary.md are generator output. Confirmed by reading the four docs.
- **SQLite single-source-of-truth (§11.4.93/§11.4.95)** — DB at docs/workable_items.db, TRACKED in git (NOT gitignored, §11.4.95). Schema read from constitution/scripts/workable-items/schema.sql: tables items (composite PK (atm_id, current_location)); columns type CHECK Bug/Feature/Task; status CHECK the 10-value closed set; severity, title, description, created_by, assigned_to, current_location CHECK Issues/Fixed, body_md, created_at, last_modified), item_history (append-only audit), obsolete_details,
 - more. Go binary at constitution/scripts/workable-items/cmd/workable-items/.
- **docs_chain engine** — at docs_chain/ (sibling, in-tree at parent HEAD). Project-agnostic Go bidirectional document/DB

propagation engine; consumers register chains as `.docs_chain/contexts/*.yaml`; inherited BY REFERENCE never copied (§11.4.80 pattern). `cmd/docs_chain/main.go` registers `sync / verify / diff / doctor`. CLI/loader IMPLEMENTED but UNTRACKED; submodule distribution (Phase 6) PLANNED + OPERATOR-GATED. Confirmed by reading `docs_chain/docs/CONSTITUTION_INTEGRATION.md` + `USE_CASE_CATALOGUE.md`.

- **Version-tag scheme** — git tags `1.1.9-dev`, `1.1.8-dev`, `1.1.5-dev-0.0.14`, ... i.e. `<major>.<minor>.<patch>-dev[-<sub>]`. Confirmed by git tag.

UNCONFIRMED / NOT FOUND (stated as fact, not invented):

- `RISK_ANALYSIS.md` / webhook-findings from a P0 research subagent are **NOT present** in this worktree (`constitution/submodules/clickup_sync/docs/research/` is empty). Where this design needs ClickUp-API specifics that a research pass would have pinned (exact webhook event names, exact rate-limit numbers, exact custom-field-type ids), this document marks them `PENDING_RESEARCH`: and the engine treats them as **config-injected constants**, never hardcoded — so the missing research does NOT block the architecture, only the concrete values the operator/research feeds it later.
-

1. New decoupled submodule `clickup_sync`

1.1 Name

Proposed: `clickup_sync` (lowercase snake_case per §11.4.29 — already the operator's chosen directory name). Generic-but-clear.

Alternatives considered: `tracker_clickup_bridge` (more generic, signals the engine is a generic tracker↔ClickUp bridge — but longer); `clickup_bridge` (shorter, drops the "sync" which is the load-bearing verb); `workitem_clickup_sync` (over-specific to the workable-items model the engine must NOT depend on). **Recommendation: keep `clickup_sync`** — the engine internals stay generic ("tracker ↔ ClickUp"); the repo name names the integration, not the consumer.

1.2 Repository

- **PUBLIC** repo under `vasic-digital` on **GitHub + GitLab** (multi-mirror per §2.1 / `install_upstreams` §11.4.36). Public so any project inheriting the constitution can consume it.

- **Language: Go** — matches docs_chain + the workable-items binary (single toolchain, shared idioms, the §11.4.93 DB layer reusable).
- Layout (§11.4.29 snake_case, §11.4.30 .gitignore, §11.4.31 helix-deps):

```

clickup_sync/
  cmd/clickup_sync/main.go      # the Go binary (all
  logic)
  internal/
    canonical/                  # union schema + mappers
  (generic)
    clickupapi/                 # ClickUp REST/webhook
  client (generic)
    trackeradapter/             # generic tracker-side
  adapter interface
    conflict/                   # source-of-truth
  resolution engine
    mapping/                    # field/status/type/label
  mappers (config-driven)
    scripts/                    # thin shell wrappers over
  the binary
    docs/design/{DESIGN.md,PLAN.md} # THIS PHASE
    docs/scripts/*.md           # §11.4.18 per-script
  companion docs
    helix-deps.yaml             # §11.4.31 dependency
  manifest
    .gitignore                  # §11.4.30
    upstreams/                  # §11.4.36
  install_upstreams recipes
    README.md / CLAUDE.md / AGENTS.md / QWEN.md # §11.4.35 +
  §11.4.65 exports

```

1.3 Decoupling (§11.4.28) — ZERO ATMOSphere specifics in the engine

The engine knows ONLY the abstract concepts "a workable item", "a ClickUp task", "a tracker adapter". **Everything project-specific is injected** via a consumer config file + env, NEVER compiled in:

Project specific	Injection point
ClickUp team / space / list IDs	CLICKUP_TEAM_ID, config target.list_id
Doc paths (docs/ Issues.md, ...)	config tracker.docs[]
ATM-NNN heading regex	config tracker.id_regex (default supplied by consumer, e.g. \[ATM-(\d+)\])
SQLite DB path	

Project specific	Injection point
	config tracker.db_path (= docs/workable_items.db)
Version-tag source	config tracker.version_tags.source: git-tags + pattern
Status / type / label mapping	config mapping.* tables (see §2)
ClickUp custom-field ids	config clickup.fields.*

The engine is therefore a generic "tracker ↔ ClickUp" bridge; ATMOSphere is just its first consumer. A second project ships a different `clickup_sync.yaml` and reuses the binary unchanged (§11.4.28(B)).

1.4 ⚠ OPEN DECISION (operator) — placement vs §11.4.28

The operator stated the convention `constitution/submodules/<name>`. This **conflicts** with §11.4.28(C): "Nested own-org submodule chains are FORBIDDEN. A submodule MUST NOT have its own `.gitmodules` entries pulling in further owned-by-us repos. Add the dependency at the parent's root path." Placing `clickup_sync` UNDER the constitution submodule makes it a nested own-org submodule of the constitution — exactly the forbidden chain.

I do **NOT** decide. Both options, with a recommendation:

- **Option A — nested under constitution/submodules/clickup_sync** (the operator's literal words). Pro: groups all "constitution-distributed" helpers in one place; matches the `constitution/scripts/*` inheritance feel; one mental model ("constitution carries the cross-project tooling"). Con: a nested own-org submodule chain — a direct §11.4.28(C) tension; the §11.4.31 dependency-manifest bridge exists precisely to AVOID this; `git submodule update --recursive` cost + the "access-from-root" rule is violated (consumers can't reach it from THEIR root, only via the constitution).
- **Option B — at project root + referenced by the constitution by-path (§11.4.28(C)-compliant).** `clickup_sync` is its own vasic-digital submodule added at `<project_root>/clickup_sync/` (or `<project_root>/submodules/clickup_sync/`); the constitution exposes it **by reference** (an env/path the constitution publishes, the exact §11.4.80 `codegraph_*` + §11.4.106 `docs_chain` pattern), NOT as a nested `.gitmodules` entry of the constitution. Pro: §11.4.28(C)-clean, accessible-from-root, same inheritance-by-reference model `docs_chain` already uses (`docs_chain` is a vasic-digital submodule consumed BY REF-

ERENCE through the constitution, NOT nested inside it). Con: the operator must place it at root, not under constitution/submodules/.

Recommendation: Option B. It is the established pattern for docs_chain (§11.4.106) and codegraph (§11.4.80) — both are own-org tooling consumed by-reference, neither nested inside the constitution. Option A re-introduces the exact nested-chain §11.4.31 was written to prevent. The Phase-0 docs already live at constitution/submodules/clickup_sync/docs/design/ per the operator's instruction for THIS deliverable; that placement of the *design docs* is orthogonal to where the *git submodule pointer* eventually lands — the operator decides the pointer location at Phase 1. **This is a §11.4.66 interactive operator decision; the build does NOT proceed past P1 scaffold until it is answered.**

1.5 Minimal-LLM (§11.4.141)

ALL sync logic lives in the Go binary. The LLM / CLI agent only GLUES and TRIGGERS (invoke clickup_sync push --dry-run, read the JSON result, decide to proceed). No per-item LLM reasoning in the hot path — token-light by construction.

2. Canonical data model + field mapping

2.1 Union schema (the canonical item)

Both sides map to/from this canonical record (a superset of the §11.4.93 items table + the ClickUp-binding columns):

```
WorkableItem {
  id                string    // ATM-NNN (canonical primary
identity)
  title             string
  description        string    // §11.4.91 floor (≥6 words / ≥40
chars)
  type              enum      // Bug | Feature | Task (§11.4.16)
  status            enum      // §11.4.15/.21/.90 closed set + NEW
"Deleted"
  severity          string    // C | M | L (informational)
  created_by        string    // canonical handle (§11.4.104)
  assignees         []string  // canonical handles
  version_labels    []string  // e.g. ["1.1.9-dev", "1.2.0-dev"] –
MULTI-VALUE
  deleted           bool      // tombstone flag
  // --- bookkeeping (NOT user-authored; sync metadata) ---
```

```

    our_rev          int          // monotonic local revision (our
last_modified-derived)
    clickup_task_id  string       // ClickUp task id binding (nullable
until first push)
    clickup_date_updated int64    // ClickUp's date_updated at last
observed sync
    last_sync_source enum        // db | docs | clickup (who
authored the last accepted change)
    sync_rev         int          // OUR sync_rev stamped into a
ClickUp custom field (echo guard)
}

```

clickup_task_id, clickup_date_updated, last_sync_source, sync_rev are **NEW sync-bookkeeping columns** the engine needs.

DESIGN NOTE: they do NOT belong in the §11.4.93 items table (that table is the project-agnostic SSoT and must stay ClickUp-agnostic per §11.4.28). They live in a **separate clickup_binding table** in the SAME DB (docs/workable_items.db), keyed by (atm_id), so the workable-items schema is untouched and the binding is an additive, removable join. This is a concrete PR target (§5 — but to the workable-items DB layer, NOT a docs_chain PR).

2.2 Mappings

Canonical	ClickUp side	Rule
type (Bug/ Feature/Task)	ClickUp custom item type	1:1 name map in config mapping.type. init-sync creates the 3 custom item types if absent (dry-run first). SAME names + SAME ORDER (column order) per config
status (closed set + Deleted)	ClickUp custom statuses	mapping.status[] (an ORDERED list). init-sync mirrors the status set AND its order onto the ClickUp list.
version_labels[] (git tags)	ClickUp tags / labels (multi-value)	A task may carry SEVERAL. mapping.version_labels.source: git-tags + pattern. New tag in our tracker → ensure ClickUp tag exists → attach.
created_by / assignees[]	ClickUp members	Resolve handle→ClickUp member id via mapping.members{} (config) + a runtime member-list lookup; STORE the resolved id in clickup_binding. Unresolvable handle → PENDING_FORENSICS: + leave unassigned, never silently drop.

Canonical	ClickUp side	Rule
id (ATM-NNN)	ClickUp custom field atm_id (text, unique)	The stable cross-side join key; ClickUp task_id is a secondary binding.
our_rev / sync_rev	ClickUp custom field helix_sync_rev (number)	Echo guard — see §3.

2.3 Status closed-set + the NEW "Deleted" status (end-to-end)

The §11.4.93 status CHECK is extended by ONE value: **Deleted** (canonical; rendered DELETED in docs). Defined end-to-end:

- **Trigger.** A deletion in ANY controlled member → mark Deleted EVERYWHERE we control:
 - DB: status='Deleted', deleted=1 (a soft-delete tombstone — NEVER a hard DELETE FROM items; history/audit must survive, §11.4.55/§9).
 - Docs: the item sinks to a bottom ## Deleted section in its source doc AND is mirrored into a NEW Deleted.md.
 - ClickUp: the task's custom status set to the Deleted column (NOT a ClickUp hard-delete — keeps the binding + audit; reconcile re-affirms).
- **New tracker docs.** Deleted.md + Deleted_Summary.md — synced + exported md / html / pdf / docx EXACTLY like Issues/Fixed (the §11.4.65 exporter is extended to add **DOCX** — see §5).
- **## Deleted bottom section.** Every primary doc (Issues, Fixed) grows a trailing ## Deleted section so a deleted-but-not-yet-archived item is visible in context before it migrates to Deleted.md. (Mirrors the §11.4.19 atomic-move discipline: an item moves Issues/Fixed → Deleted.md in one commit.)
- **Constitution impact.** Adding Deleted to the §11.4.93 status CHECK + §11.4.15 closed-set is a **constitution-submodule change** governed by §11.4.26 (fetch-first → edit → validate → push-all → bump pointer) and is a §11.4.66 operator decision (closed-set vocabulary changes are operator-gated). FLAGGED as a P6 operator-gated step, NOT done here.

3. Conflict resolution / source-of-truth

The P0 research subagent's RISK_ANALYSIS.md is NOT present in this worktree (§0). The mechanism below is derived from first principles + the ClickUp API's documented date_updated

field; the concrete rate-limit / webhook-event constants are PENDING_RESEARCH: and config-injected. The ALGORITHM is complete and does not depend on those constants.

3.1 The echo-guard (don't mistake our own write for a user edit)

Every write the engine makes to ClickUp stamps the ClickUp custom field `helix_sync_rev` = `<our sync_rev>`. On any inbound ClickUp change (webhook or poll), the engine compares the task's `helix_sync_rev` against the `sync_rev` we last wrote:

- `clickup.helix_sync_rev == our last-written sync_rev AND clickup.date_updated` within the echo window → **this is the echo of our own write** → ignore (idempotent no-op).
- otherwise → a genuine user-side edit → candidate for pull.

Symmetrically, every write the engine makes to our DB/docs bumps `our_rev`; a ClickUp pull that would re-apply a value already at `our_rev` is a no-op.

3.2 "Which is newer" algorithm (deterministic, stated as an algorithm)

```
resolve(item):
    ours_changed    = item.our_rev                >
item.our_rev_at_last_sync
    theirs_changed = clickup.date_updated        >
item.clickup_date_updated
                        AND NOT is_echo(clickup)      # §3.1 echo
guard

    if ours_changed and not theirs_changed: WINNER = ours      #
push
    if theirs_changed and not ours_changed:  WINNER = theirs   #
pull
    if not ours_changed and not theirs_changed: WINNER = none  #
no-op
    if ours_changed and theirs_changed:                        #
TRUE CONFLICT
        # field-level merge first (disjoint fields auto-merge),
        # then for genuinely-colliding fields apply config policy:
        #   policy.conflict.source_of_truth ∈ {tracker | clickup |
newest-wins | operator}
        #   default = "operator": mark item status-note CONFLICT,
        #   write BOTH values to a conflict record,
BLOCK auto-write,
        #   surface via §11.4.66 (never silently pick a
side, never lose data).
```


is_echo is the §3.1 guard. newest-wins compares the two monotonic timestamps. The DEFAULT is operator (no silent data loss) — the safest choice per §11.4.101 (irreversible-on-a-side conflict → block + ask).

3.3 Idempotency + transactional guarantees (no loss / no corruption / no desync)

- **Local writes** go through the workable-items DB layer in a single SQLite transaction (BEGIN ... COMMIT); WAL mode (per the existing schema) → crash mid-write rolls back cleanly (chaos test mid-write-SIGKILL, §7).
- **Doc regeneration** is docs_chain's atomic-rename + DB-txn commit + rollback (§11.4.106) — never a partial doc write.
- **ClickUp writes** are idempotent by (atm_id, helix_sync_rev): re-sending the same sync_rev is a documented ClickUp update that produces the same end-state (PUT-semantics, not POST-append). Each push records the resulting clickup_date_updated so a re-run sees no change.
- **Ordering.** A sync run is: (1) snapshot both sides, (2) compute the resolution set in memory, (3) apply LOCAL writes in one DB txn, (4) apply ClickUp writes one-by-one recording each result, (5) commit the binding table LAST. A crash before step 5 leaves the binding unchanged → next run re-derives the same set (idempotent replay).
- **Reconcile backstop.** reconcile (full sweep) re-reads both sides end-to-end, recomputes the union, and repairs ANY drift webhooks/incremental runs missed. This is the desync safety-net (cron, §6).
- **§9 data-safety.** Any operation that could lose history (e.g. a ClickUp hard-delete) is converted to a soft-delete tombstone; a genuinely destructive op requires a §9.2 hardlinked pre-op DB backup + operator authorization. NO force-push anywhere (§11.4.113).

4. Sync engine surface (scripts/ — Go binary + thin shell wrappers)

Every command defaults to **--dry-run** (prints the plan + the would-be writes, touches nothing) so ClickUp is NEVER polluted with trash. A real run requires explicit **--apply** (or **--no-dry-run**).

Command	Purpose	Notes
init-sync	Mirror our status-set / item-types / version-labels /	Dry-run first → prints the diff → --apply creates/reorders. Result-validation step: after apply, re-reads the ClickUp list

Command	Purpose	Notes
	column ORDER onto the ClickUp list.	and asserts statuses+order+types match OUR System (anti-bluff: a faked "init done" with the wrong column order FAILs the validation).
pull	ClickUp → our DB/docs (read-only on ClickUp).	Safe; respects echo-guard.
push	our DB/docs → ClickUp.	--dry-run default.
bidirectional	one reconciled pass both directions (§3 algorithm).	The normal incremental op.
reconcile	full sweep both sides, repair all drift.	The cron/backstop op; idempotent.
webhook-serve	event-driven receiver (signature-verified, deduped).	§6. Long-running service.
self-sync	the cron entry point (calls bidirectional then a periodic reconcile).	systemd-timer / cron.

Thin shell wrappers (`scripts/clickup_sync_*.sh`) just exec the Go binary with the resolved config (so the §11.4.96-SAFE catalogue + the `clickup-sync` skill / MCP can call a stable shell surface). All wrappers parse cleanly under `sh -n` AND `bash -n` (§11.4.67).

5. docs_chain extension scope (concrete PRs to docs_chain)

`docs_chain` is the §11.4.106 canonical doc/DB sync engine; `clickup_sync` does NOT re-implement doc generation — it EXTENDS `docs_chain`. Concrete PRs:

- **PR-DC-1 — DOCX export target.** Add a docx output format to the §11.4.65 exporter (`pandoc --to docx / weasyprint-equivalent`), wired so every chain that emits `md/html/pdf` can also emit `docx`. Honest `ToolAbsentError` + `SKIP` if `pandoc/docx` backend

absent (§11.4.106 no-fake). Needed for Issues/Fixed/Deleted four-format exports.

- **PR-DC-2 — Deleted doc class.** A new chain-node recipe in the USE_CASE_CATALOGUE for Deleted.md + Deleted_Summary.md (mirrors recipe (a)/(b) Issues/Fixed chains) — DB deleted=1 rows → Deleted.md → summary → md/html/pdf/docx.
- **PR-DC-3 — ClickUp chain-node type.** A new docs_chain member KIND clickup (alongside the existing markdown / db members) so a chain can declare ClickUp as a synced endpoint and docs_chain's sync/verify/diff drive the clickup_sync binary as the transform. (DESIGN NOTE: alternatively clickup_sync stays OUTSIDE docs_chain and only the doc-side is a docs_chain chain — see PR-DC-5; the operator chooses coupling depth.)
- **PR-DC-4 — metadata-stamping transform.** A transform primitive that writes helix_sync_rev / our_rev round-trip metadata without polluting the human-authored body (so the echo-guard fields survive db↔md round-trips byte-identically per §11.4.93).
- **PR-DC-5 (decoupling-safe alternative to PR-DC-3).** Keep ClickUp OUT of docs_chain; docs_chain owns ONLY the local doc/DB fan-out, clickup_sync owns the ClickUp leg and CALLS docs_chain for the doc side. Smaller blast-radius, keeps docs_chain ClickUp-agnostic. **Recommended** unless the operator wants ClickUp as a first-class docs_chain member.

Separate (NOT a docs_chain PR): the clickup_binding table + the Deleted status value land in the **workable-items DB layer** (constitution/scripts/workable-items/schema.sql) — additive columns/table, operator-gated for the status CHECK change (§11.4.26).

6. Live events (webhooks) + cron backstop

Concrete ClickUp webhook event NAMES + the signature header + exact retry/replay semantics are PENDING_RESEARCH: (the research subagent's webhook findings are not in this worktree, §0). They are config-injected constants; the design is event-name-agnostic.

- **Webhook receiver (webhook-serve).** Subscribes to ClickUp task create/update/delete/status-change/comment events (exact event tokens PENDING_RESEARCH:, config clickup.webhook.events[]). On each event:
 1. **Signature-verify** the request (ClickUp HMAC secret in CLICKUP_WEBHOOK_SECRET, §11.4.10 env-only). Reject unsigned/invalid.

2. **SSRF-hardened** — the receiver makes NO outbound request to a URL derived from the payload; it only re-reads the named task from the ClickUp API base (CLICKUP_API_BASE, fixed). (Security test §7.)
 3. **Dedupe** by (event_id) — ClickUp may redeliver; a seen event_id within the dedupe window is a no-op (idempotent).
 4. **Echo-guard** (§3.1) — drop our own write's echo.
 5. Enqueue a targeted bidirectional --item <atm_id> pass.
- **Cron self-sync backstop.** A systemd-timer / cron entry runs self-sync on a cadence (default hourly bidirectional + a daily reconcile full sweep) so that when webhooks miss / the receiver was down / ClickUp dropped a delivery, the next sweep repairs all drift. The reconcile sweep is the §3.3 desync safety-net.
-

7. Test plan (100%-per-type, all --dry-run / sandbox, anti-bluff)

All tests run against a **sandbox/trash-free path** — a dedicated ClickUp sandbox list id (CLICKUP_SANDBOX_LIST_ID) or a recorded/mock ClickUp fixture; NEVER the production list. Captured evidence per §11.4.69/§11.4.107 under qa-results/clickup_sync/<run-id>/.

Type	Coverage
unit	mappers (type/status/label/member), the §3.2 resolve() truth-table, echo-guard, idempotency, ATM-NNN regex, version-tag parse. (Mocks allowed ONLY here, §11.4.27.)
integration	real binary ↔ sandbox-ClickUp (or recorded fixture) round-trip: push→pull→assert canonical equality; init-sync dry-run→apply→result-validation.
e2e	full bidirectional over a seeded item set: create in DB → push → edit in ClickUp sandbox → pull → assert; the Deleted lifecycle DB→docs→ClickUp→Deleted.md. Fully automated, re-runnable -count=3 (§11.4.98).
stress	≥100-item sweep latency p50/p95/p99; ≥10 concurrent bidirectional invocations (no deadlock, no double-write).
chaos	internet-down mid-sync (partial-apply → reconcile repairs); 429-storm (backoff + no data loss); webhook replay (dedupe holds); concurrent multi-user edit (true-conflict → operator-block, no silent loss); mid-write SIGKILL (DB txn rollback); corrupt-DB recovery (restore from §9.2 backup). Cleanup in trap ... EXIT (§11.4.14).

Type	Coverage
security	SSRF on the webhook receiver (payload-supplied URL must NOT be fetched); signature-bypass attempt rejected; secret never logged (§11.4.10); replay-token reuse rejected.
perf / bench-mark	per-item sync cost, webhook handler latency budget.
Challenges + HelixQA	a clickup-sync Challenge bank entry + an autonomous HelixQA session driving the dry-run surface, scoring PASS only on captured result.json (§11.4.27/§11.4.116 sync channel).

Every PASS cites a captured-evidence artefact (§11.4.69 `ab_pass_with_evidence`); the `resolve()-truth-table` + the SSRF test each carry a paired §1.1 meta-test mutation (flip a cell / drop the SSRF guard → test FAILs). No test mutates production ClickUp — the `--dry-run` default + `sandbox-list` guard is the mechanical floor.

8. Skills / Plugins / MCP (thin glue, token-light §11.4.141)

- **clickup-sync Claude Code skill** — a thin skill whose body is a short command index (`clickup_sync init-sync --dry-run, ... bidirectional, ... reconcile`) + the anti-bluff reminders; it does NOT carry sync logic (that's the Go binary). Loads out-of-the-box; inherited by every CLI agent via the §11.4.35 context carrier.
- **clickup-sync MCP server** — exposes the binary's commands as MCP tools (`clickup_sync.dry_run_plan, clickup_sync.reconcile, ...`) so any MCP-capable agent (Claude Code, Cursor, Qwen, ...) can trigger a sync and read the JSON result without shell access. Stdio MCP, project-scoped config where supported, host-local otherwise; references the bare `clickup_sync` binary on `PATH` (no hardcoded path, §11.4.78 pattern).
- **Token-light**: the skill + MCP carry only the command surface; all reasoning output is persisted to a file (`qa-results/...`) not an inline transcript (§11.4.141).

9. "No ClickUp env → integration disabled" guard (§11.4.96 latent-inherit)

Every gate / script / rule / cron / webhook no-ops cleanly when the `CLICKUP_*` env (minimum `CLICKUP_API_TOKEN` + `CLICKUP_TEAM_ID`) is unset:

- The Go binary's first action is a `config.Detect()` — if the required env is absent it prints `clickup_sync: DISABLED` (no `CLICKUP_*` env) and exits 0 (a clean no-op, NOT an error).
 - Pre-build / CI gates that touch `clickup_sync` are written as "SKIP-with-reason `feature_disabled_by_config`" (§11.4.69) when the env is unset — never FAIL, never PASS-by-default.
 - The cron self-sync / webhook-serve units are conditional on the env; absent → not started.
 - So a NON-ClickUp project that inherits the constitution carries the submodule LATENTLY (binds the moment it ships `CLICKUP_*` env), exactly the §11.4.96 / §11.4.104 latent-inheritance pattern.
-

10. Cross-references

- `PLAN.md` (sibling) — the ordered phased plan with per-phase 4-layer test gates + effort estimates.
- Constitution anchors composed: §11.4.6 (no-guessing), §11.4.10 (credentials), §11.4.14 (cleanup), §11.4.26 (constitution-update), §11.4.27 (no-fakes-beyond-unit), §11.4.28 (decoupling / no-nested-chains), §11.4.29 (`snake_case`), §11.4.30 (`.gitignore`), §11.4.31 (dependency manifest), §11.4.35 (canonical-root inheritance), §11.4.36 (`install_upstreams`), §11.4.65 (universal MD export — extended w/ DOCX), §11.4.66 (operator decision), §11.4.67 (shell parseability), §11.4.69 (sink-side evidence), §11.4.85 (stress+chaos), §11.4.93/§11.4.95 (workable-items DB SSoT), §11.4.96 (latent inherit), §11.4.101 (autonomous-decision / conflict-block), §11.4.106 (`docs_chain`), §11.4.107 (liveness evidence), §11.4.113 (no force-push), §11.4.141 (token-light), §9 / §9.2 (data safety).